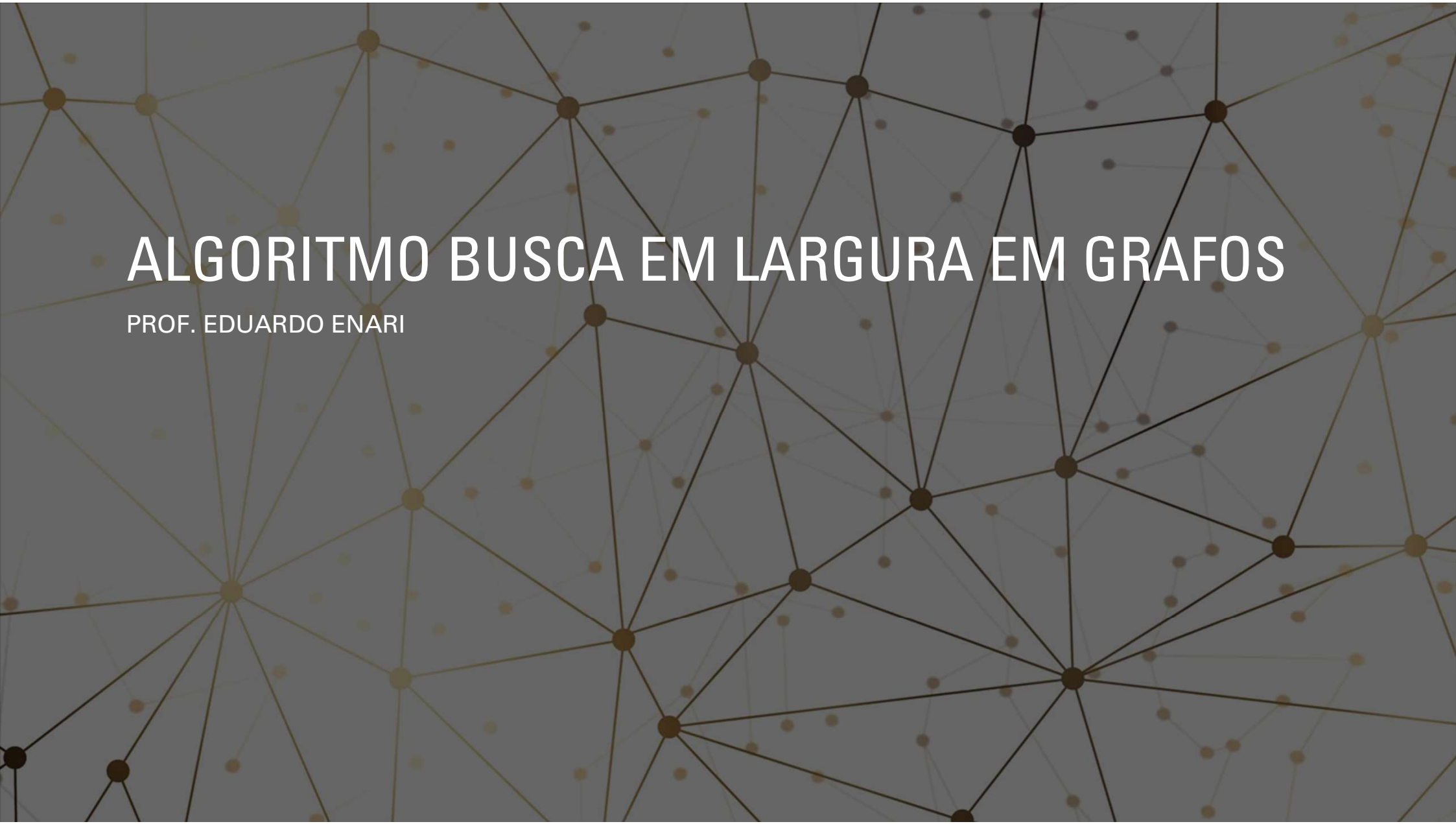


ALGORITMO BUSCA EM LARGURA EM GRAFOS

PROF. EDUARDO ENARI

OBJETIVO

- Apresentar o algoritmo busca em largura sobre grafos.

BUSCA EM LARGURA

- Variação de algoritmo de busca em grafos que também realiza o caminhamento sobre componentes conexos do grafo.
- Ao invés de avançar para vértices mais distantes do vértice inicial, realiza a visitação de todos os vértices adjacentes ao vértice inicial, para continuar o caminhamento a partir dos vértices adjacentes de seus vizinhos.
- Em outras palavras, o algoritmo visita todos os vértices que estão a uma aresta de distância do vértice inicial para a partir daí visitar todos os vértices que estão a dois vértices de distância do vértice inicial.
- Esse processo de expansão da distância de busca continua até que não hajam mais vértices a serem visitados no componente conexo.

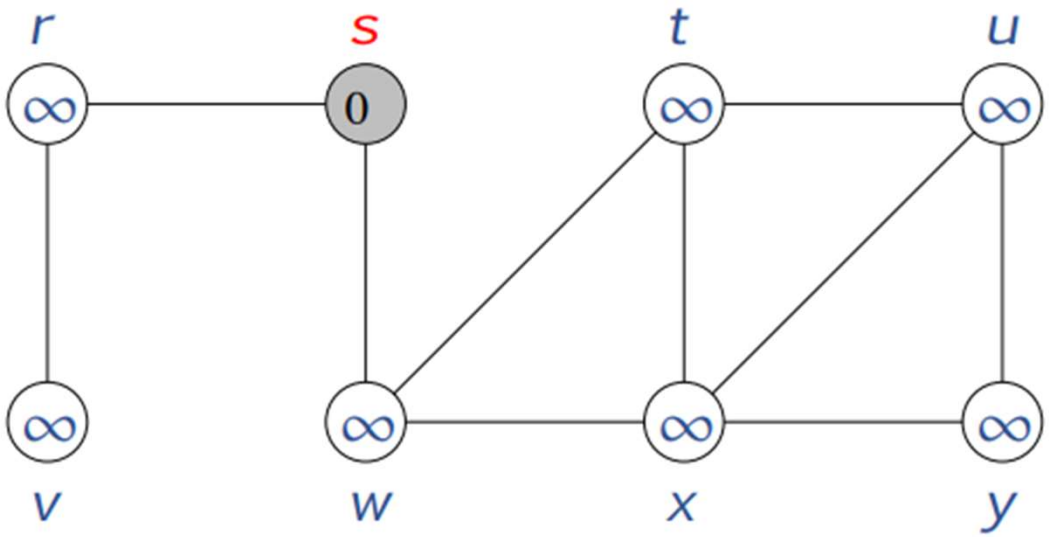
BUSCA EM LARGURA

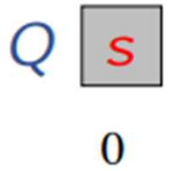
- Algoritmo geral:
  - Inicia-se de um vértice origem (s).
  - Registra-se todo vizinho v, adjacente à s.
  - O processo se repete para cada vizinho v de s, até que todos os vértices atingíveis a partir de s sejam visitados.
- Todo vértice registrado é armazenado em uma fila Q.
- O vértice que sai da fila é o vértice a partir do qual se visitarão todos os vértices vizinhos.
- O processo termina quando a fila estiver vazia.

BUSCA EM LARGURA

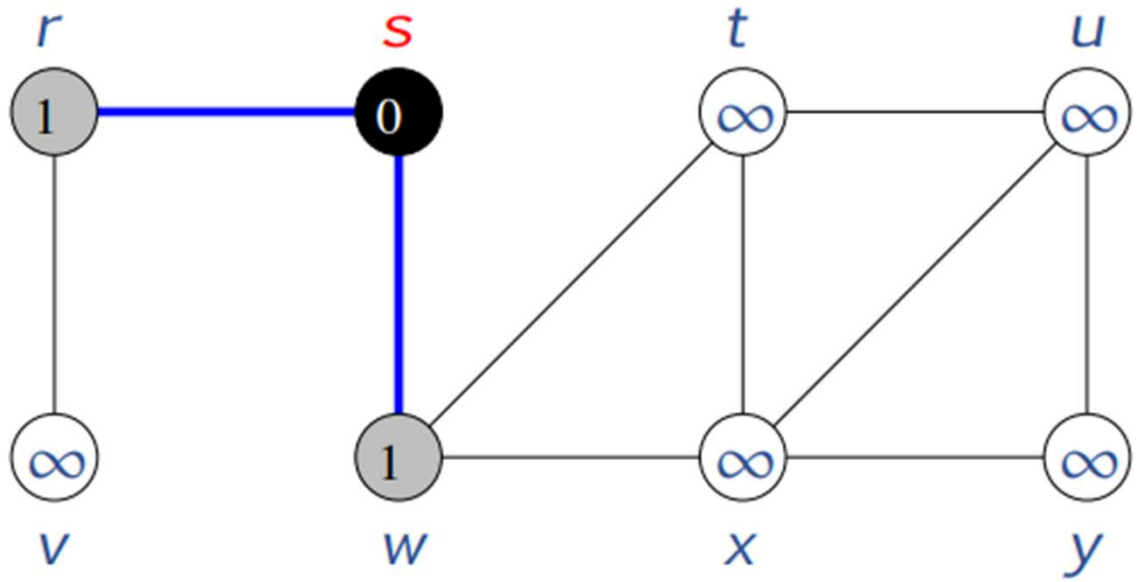
- A busca em largura atribui cores a cada vértice:
  - Branco = “não visitado”
  - Cinza = “descoberto”
  - Preto = “teve seus vizinhos visitados”

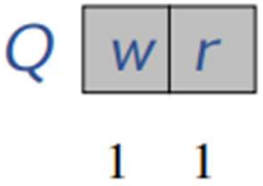
EXEMPLO:



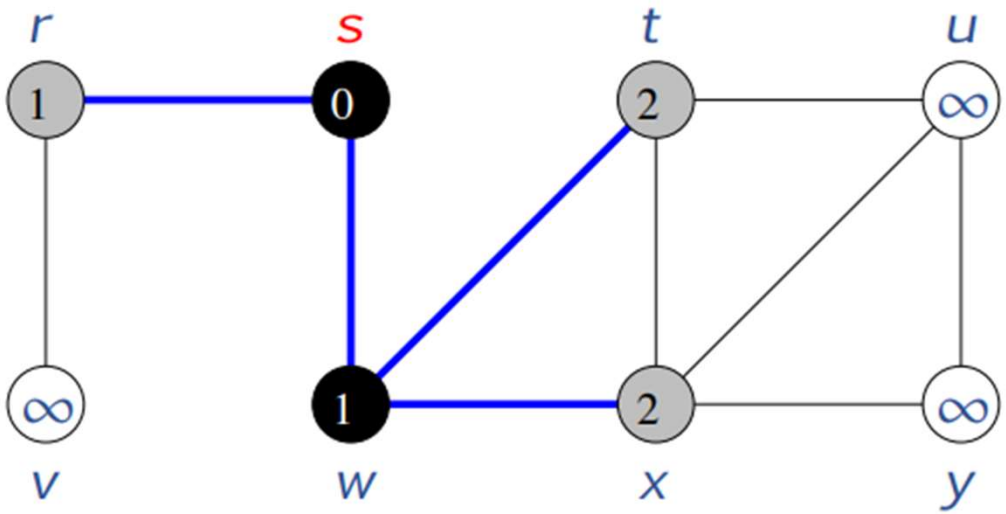


EXEMPLO:



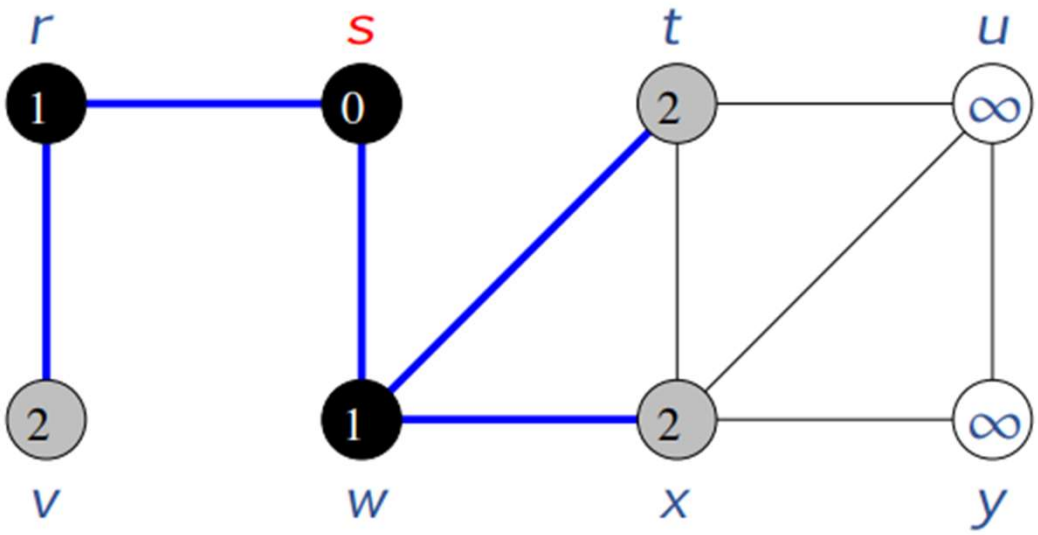


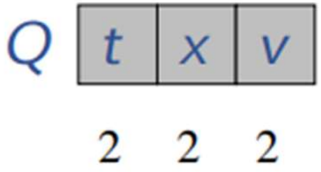
EXEMPLO:



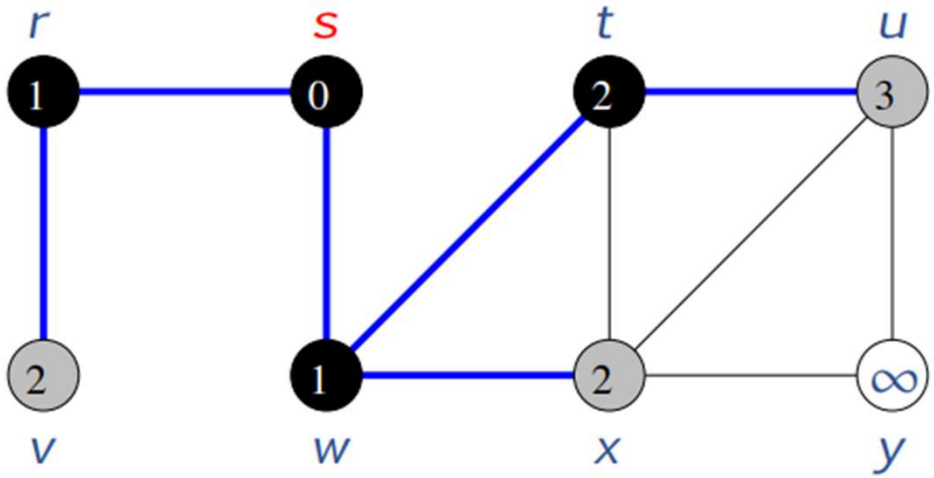


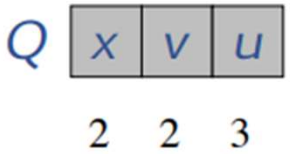
EXEMPLO:



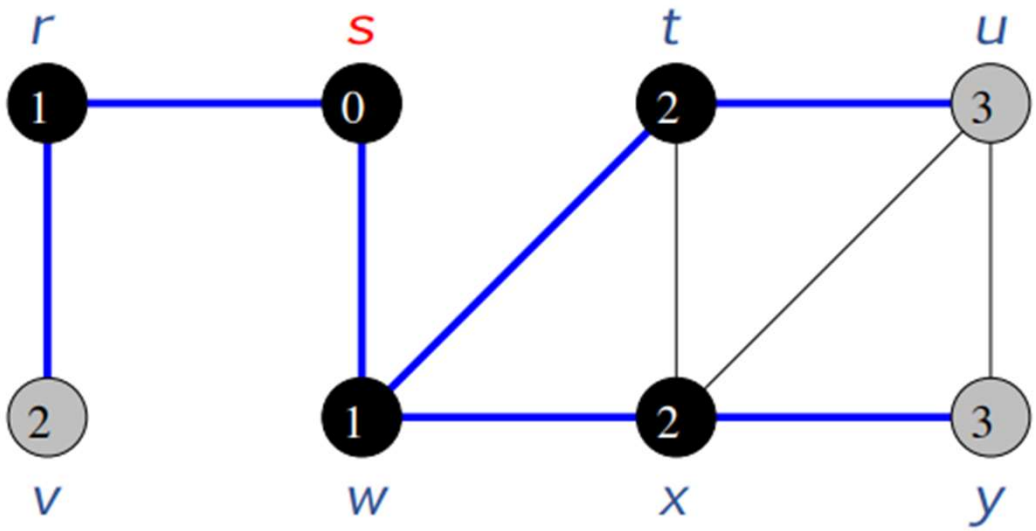


EXEMPLO:



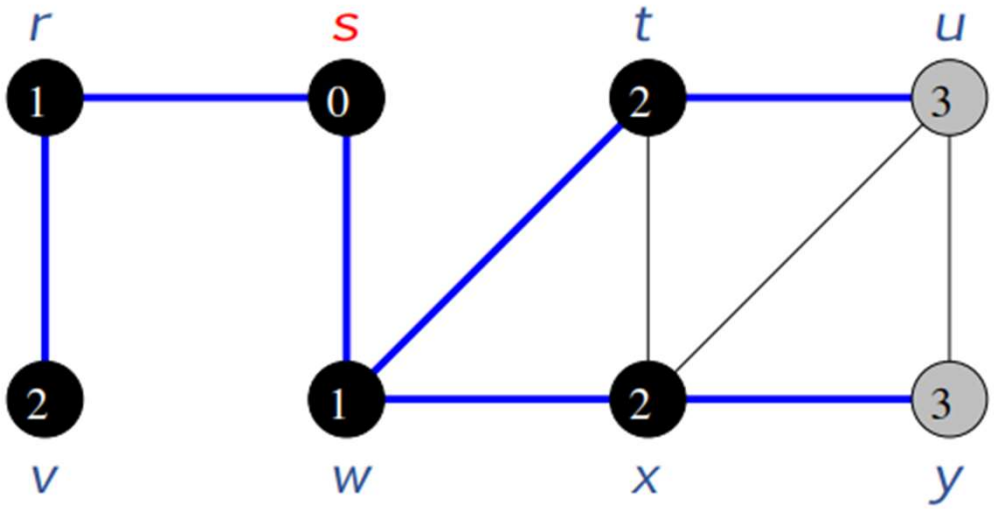


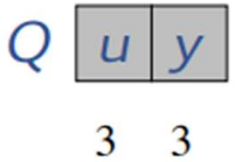
EXEMPLO:



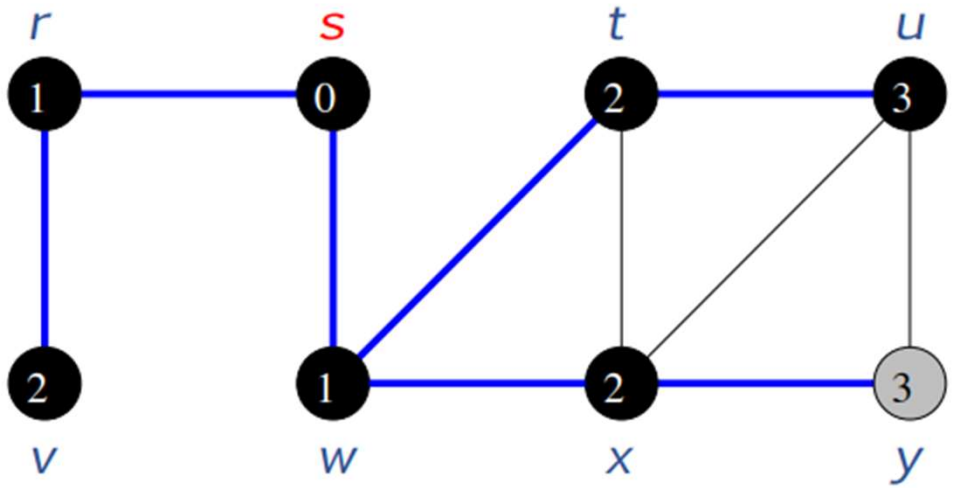


EXEMPLO:





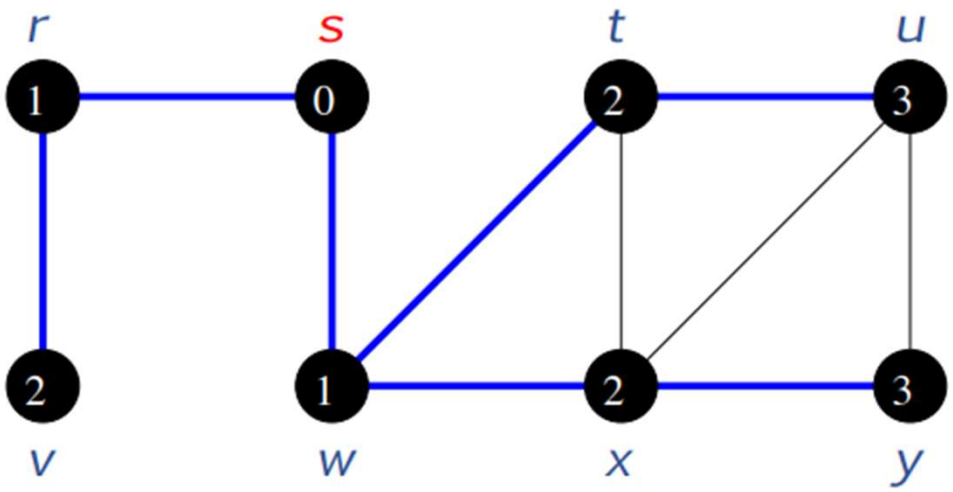
EXEMPLO:



Q | y

3

EXEMPLO:



Q ∅

CONSIDERAÇÕES SOBRE O ALGORITMO

- Pode-se utilizar um vetor **cor[v]** que pode armazenar a cor de cada vértice que pode ser **branco**, **cinza** ou **preto**
  - É possível implementar sem essa marcação de cores, mas pode facilitar o entendimento do funcionamento do algoritmo, se apresentado a cada passo.
- Pode-se armazenar a sequência de nós visitados de duas formas:
  - Um vetor d[v] que armazena a distância (em quantidade de arestas) do vértice **s**, sendo d**[s] = 0**.
  - Um vetor pr[v] que armazena em cada posição o índice do próximo vértice a ser visitado.
- Assim, uma busca em largura recebe um Grafo G e um vértice s
- Retorna para cada vértice v do Grafo G a distância de **s** a **v** em **d[v]** e uma árvore de Busca em Largura com raiz em **pr[s] (pr[s] => pr[?])**

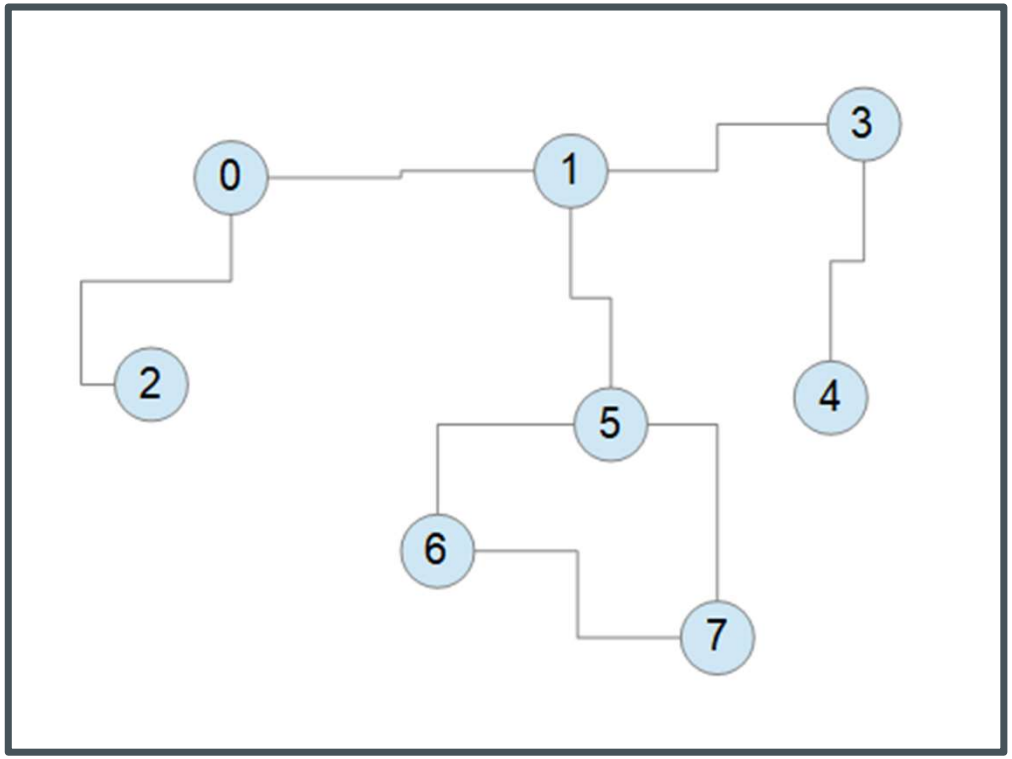
BUSCA EM LARGURA (INICIALIZAÇÃO)

```
BFS(G, s)
    para cada vértice u de  G faça
        cor[u] = branco
        d[u] = ∞
        pr[u] = NIL
    cor[s] = cinza
    d[s] = 0
    pr[s] = nil
    fila.push(s)
```

BUSCA EM LARGURA (ROTINA PRINCIPAL)

```
enquanto !fila.empty() faça
    u = fila.pop();
    para cada vértice v vizinho de u faça
        se cor[v] = branco então
            cor[v] = cinza
            d[v] = d[u] + peso(u, v)
            pr[v] = u
            fila.push(v)
    cor[u] = preto
```

EXERCÍCIO 1:



- Implemente o algoritmo de busca em largura para a Classe Grafo que armazena o grafo em uma matriz de adjacências.
- Realize teste do algoritmo sobre o Grafo que foi utilizado no teste da busca em profundidade
- Modifique o grafo para que se torne direcionado e teste a sua implementação.

EXERCÍCIO 2:

- Modifique a implementação da busca em largura do exercício 1 para que seja realizada a visitação de todos os vértices do grafo, mesmo em grafos não conexos.
- Modifique o grafo do teste da implementação, para testar a sua implementação (torne o grafo não conexo – utilize o mesmo grafo que você construiu para o exercício 1 da busca em profundidade e compare os resultados).
- Faça um teste da implementação sobre um grafo direcionado.

EXERCÍCIO 3:

- Implemente a busca em largura na classe GrafoLA, que utiliza Lista de Adjacência para armazenar o Grafo.
- Execute os mesmos testes que foram usados no Grafo com Matriz de Adjacências.

EXERCÍCIO 4

- Construa um método qtdComponentesConexos() nas classes Grafo e GrafoLA, que retorna um número inteiro correspondente à quantidade de componentes conexos em um grafo não direcionado, com a ajuda da implementação da busca em largura.